SIH 2026 - Master Q&A & Architectural Deep Dive

This document provides an uncompromising, highly technical breakdown of the SIH 2026 SignKYC project. It is designed to preemptively answer the most brutal architectural, algorithmic, and systemic questions judges can throw at you.

---

1. The Machine Learning Models & Tech Stack

Q: What exact Machine Learning models are you using?

A: Our pipeline relies on three distinct types of models, each chosen for a specific trade-off between speed, privacy, and accuracy:

1. **Pose Estimation Model:** `pose_landmarker_full.task` (Float16 precision). We use Google's MediaPipe Tasks Vision (`@mediapipe/tasks-vision@0.10.21`). We explicitly chose the `full` tier (rather than `lite` or `heavy`) and `float16` quantization to strike the perfect balance between high-fidelity 33-point tracking and real-time 30fps browser execution via WebAssembly/GPU delegates.
2. **Hand Tracking Model:** `hand_landmarker.task` (Float16 precision). This tracks 21 3D coordinates per hand. Like the pose model, it runs locally in the browser to ensure zero video frames are ever sent to a server.
3. **Gesture Classification (Algorithmic):** We do **not** use an LSTM, Transformer, or CNN for the actual sign classification. Instead, we use a custom **Dynamic Time Warping (DTW)** algorithm acting as a K-Nearest Neighbor (K=1) classifier over a 106-dimensional feature space. We chose DTW over Neural Networks because DTW allows "Zero-Shot" or "One-Shot" learning—we can add a new sign to the dictionary instantly with a single `.npy` file without needing to retrain a massive neural network dataset.
4. **Natural Language Processing (NLP) Model:** `gemini-2.0-flash`. For the final step of translating ISL syntax (Subject-Object-Verb) into grammatically correct English (Subject-Verb-Object), we use Google's Gemini 2.0 Flash via the `@google/genai` SDK. It is fast, highly capable of understanding grammar contexts, and cost-effective.

Q: What is your exact Tech Stack?

A:

- **Frontend:** React 18, TypeScript, Vite (for ultra-fast esbuild bundling), TailwindCSS (for styling), and Lucide React (for icons).
- **Computer Vision:** MediaPipe Holistic (TypeScript/WASM for production, Python/OpenCV for offline template recording).
- **Backend / API:** Node.js, Express.js.
- **Database / Auth:** Supabase (PostgreSQL with Row Level Security).
- **AI Integration:** Google GenAI SDK (`gemini-2.0-flash`).

---

2. System Architecture & The "Graphify" View

Q: From a high-level component view, how is your entire system architected?

A: Our system is a highly decoupled pipeline consisting of four major hyper-components:

1. **The Python Offline Template Generator (`isl_dtw/utils.py`):** Used strictly for data collection. We use Python, OpenCV (`cv2`), and MediaPipe to record 106-dimensional `.npy` templates. We intentionally keep Python completely out of the online production loop to avoid heavy backend GPU compute costs.
2. **The Typescript WebAssembly Inference Engine (`holisticLandmarker.ts` & `dtwWorker.ts`):** We rewrote the exact Python feature extraction logic 1:1 in TypeScript. This runs entirely in the customer's browser via WASM/WebGL, ensuring zero-latency and maximum privacy.
3. **The State Machine Middle-Tier (`sentenceEngine.ts`):** A strict FSM (Finite State Machine) that buffers real-time gesture streams, deduplicates them, and acts as the gatekeeper to the cloud.
4. **The Cloud API & LLM Tier (`api/index.ts` & `geminiPool.ts`):** An Express.js backend that handles the heavy lifting of translating ISL syntax to English via Gemini 2.0 Flash, protected by our custom resilient API pool.

Q: Why do you have both a Python pipeline and a TypeScript pipeline?

A: Symmetry and Scalability. Python is the industry standard for ML data wrangling. We use it to quickly record, clean, and validate our `.npy` ISL templates. However, deploying a Python OpenCV backend to process live 30fps webcam feeds for millions of concurrent users is financially impossible and has massive network latency. By translating the exact extraction math (the 106-dim wrist/mid-shoulder normalization) into a TypeScript Web Worker, we

offload **100% of the compute** to the customer's device. The backend only handles lightweight API requests, making our system infinitely scalable at practically zero cost.

---

3. The Sentence State Machine (`sentenceEngine.ts`)

Q: How do you prevent the AI from spamming the backend with every single recognized frame?

A: We built a custom Finite State Machine (`sentenceEngine.ts`) that manages the flow of tokens. It operates on four states: `IDLE → ACCUMULATING → DISPATCHING → WAITING_RESPONSE`.

- **Deduplication:** When the DTW engine recognizes a gesture, the engine checks the `tokenBuffer`. If the new gesture is a consecutive duplicate (e.g., the user holds the sign for "Apple" for 2 seconds), the engine instantly drops the duplicate token.
- **Configurable Delimiters:** We allow the user/official to configure a specific "STOP" gesture (persisted via `localStorage`). When this gesture is detected, the FSM instantly transitions to `DISPATCHING` and fires the accumulated tokens to the backend.
- **Auto-Dispatch:** If the user forgets to sign "STOP", the engine utilizes a debounced `idleTimer`. After exactly 18 seconds (`IDLE_TIMEOUT_MS = 18000`) of no new tokens, it auto-dispatches the sentence to ensure the conversation never stalls.

---

4. The DTW Engine & Computer Vision

Q: Explain the exact math and complexity behind your Web Worker DTW algorithm.

A: Standard DTW has an `O(N * M)` time complexity and requires an expensive `Math.sqrt()` per matrix cell, which would freeze the browser.

1. **Sakoe-Chiba Band:** We apply a Sakoe-Chiba constraint (radius `r = 5`). This restricts the search space strictly around the diagonal of the cost matrix, dropping time complexity from `O(N * M)` to **`O(N * r)`**.
2. **Squared Euclidean:** We calculate distance as `(x1 - x2)² + (y1 - y2)²`. Dropping `Math.sqrt()` eliminates ~540,000 square root calls per second.
3. **Zero Heap Allocation:** Standard DTW allocates a massive `N x M` matrix, causing UI

stutter when the Garbage Collector (GC) runs. We optimized space complexity to `O(M)` using two pre-allocated 1D arrays (`dtwPrevRow` and `dtwCurrRow`). We use a ring buffer (`ringBuffer`) and a pre-allocated snapshot array (`snapshotBuffer`), meaning **zero arrays are allocated during the `matchGesture` loop**.

4. **Downsampling:** While the UI renders at a buttery 30fps, the DTW engine purposely drops every other frame (15fps) for evaluation, halving the compute load with zero loss in recognition accuracy.

---

5. The 4-Tier API Resilience Strategy (`geminiPool.ts`)

Q: Free tier APIs fail all the time during live demos. How did you handle Gemini rate limits (429s)?

A: We engineered a highly aggressive, resilient `GeminiKeyPool` .

1. **Multi-Key Round-Robin:** We inject 4 different Gemini API keys into our environment (`GEMINI_API_KEY` through `GEMINI_API_KEY_4`). The pool rotates across them seamlessly.
2. **Token Bucket Tracking:** The free tier allows 15 RPM (Requests Per Minute). Our pool tracks every timestamp within a 60-second sliding window and hard-caps each key at **13 RPM** to prevent exhaustion.
3. **429 Cooldown & Backoff:** If a key does throw a 429 Resource Exhausted error, the engine parses the `Retry-After` header, places that specific key in a 65-second "penalty box," and instantly attempts the next key. It uses exponential backoff (`Math.min(BASE_BACKOFF * 2^attempt, MAX_BACKOFF)`) if all keys are hot.
4. **LRU Cache & In-Flight Deduplication:** We cache responses with a 1-hour TTL. More importantly, we do **in-flight deduplication**. If multiple identical API requests fire simultaneously, the pool returns the *same pending Promise* for all of them, only hitting the actual Gemini endpoint once.
5. **Graceful Degradation:** If the internet drops or all keys fail completely, the API catches the exception and returns the raw concatenated tokens rather than throwing a 500 server error, ensuring the React UI never crashes.

---

6. Security & RBI Compliance (V-CIP)

Q: How do you prove this is compliant with RBI's V-CIP (Video Customer Identification Process) guidelines?

A:

1. **Liveness Detection:** In our `VideoPanel.tsx` (Step 5/7 of the KYC loop), the customer is prompted with a dynamic 4-digit Liveness Code. They must sign these digits in exact order. Our state machine tracks the `livenessMatchIndex` and visually verifies the liveness challenge.
2. **Concurrent Audit Persistence:** Using **Supabase**, we store these verified states in the `kyc_sessions` table. Because Supabase is native PostgreSQL, we enforce strict **Row Level Security (RLS)** policies (`Allow public insert to kyc_sessions` with specific `WITH CHECK` clauses) directly in `supabase_schema.sql`. This creates an immutable, timestamped audit log of the session, the liveness match, and the human-in-the-loop interpreter confirmation.
3. **Path Traversal Security:** Our custom template Express backend (`api/index.ts`) explicitly sanitizes the `req.params.filename`. It strictly checks for the `.npy` extension and rejects any path containing `..`, `/`, or `\`, preventing Directory Traversal attacks that could expose server source code.

---

7. Concurrency, Cross-Origin Isolation, and Data Collection

Q: Your Web Worker needs to process video frames incredibly fast. How are you avoiding memory copy overhead between the main thread and the worker?

A: We explicitly configured our Express server (`server.ts`) to inject strict **Cross-Origin Isolation** headers (`Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp`). By forcing the browser into an isolated context, we unlock the ability to use `SharedArrayBuffer`. This allows our `dtwWorker.ts` and the main UI thread to write and read from the exact same block of RAM concurrently, completely bypassing the expensive serialization/deserialization overhead of standard `postMessage` calls.

Q: How did you efficiently collect the training data for your templates?

A: We built a rigorous in-browser collection tool (`TemplateRecorderModal.tsx`). Because recording in Python often creates slight discrepancies compared to browser-based inference,

we built a UI that directly taps into the WebAssembly pipeline. It uses an automatic ring-buffer capture mechanism (`MAX_FRAMES = 40`) to record exactly 10 variations (`TEMPLATES_PER_CLASS = 10`) of a sign in rapid succession, automatically encoding them into `.npy` format and POSTing them to the backend. This proves the system is capable of rapid "few-shot" data onboarding natively.

---

8. Presentation Safety & Demo States

Q: Live demos usually break. What happens if the live recognition fails during the SIH pitch?

A: We designed a robust "Demo State" multiplexer directly into `App.tsx`. Because live lighting or network conditions can be unpredictable on a convention floor, the presenter can manually force the FSM into specific presentation scenarios (`normal_recognition` , `liveness_code_step` , `low_confidence_or_escalated` , `session_complete`). This allows us to seamlessly demonstrate complex edge cases—like gracefully bridging a certified human interpreter (Ananya M.) into the call if the AI confidence drops below a threshold—without praying for a live failure. Additionally, the entire conversation log is safely flushed to `localStorage` (`signkyc_session_conversation_log`) to survive unexpected browser refreshes.

---

9. Closing Statement Strategy

Q: "Your project is built with a lot of hackathon shortcuts. How is this production-ready?"

A: "While we used rapid-prototyping tools like Vite (for native ES module hot-reloading) and Supabase (to bypass backend CRUD boilerplate), the **core algorithmic pipeline is production-grade**. Our DTW engine achieves `O(N*r)` time complexity with zero heap allocation and zero-copy memory via `SharedArrayBuffer` . Our feature vectors are strictly translation-invariant, and our AI routing utilizes an enterprise-style multi-key resilient pool with in-flight deduplication. Moving this to a production AWS environment is just a matter of changing environment variables, not rewriting logic."